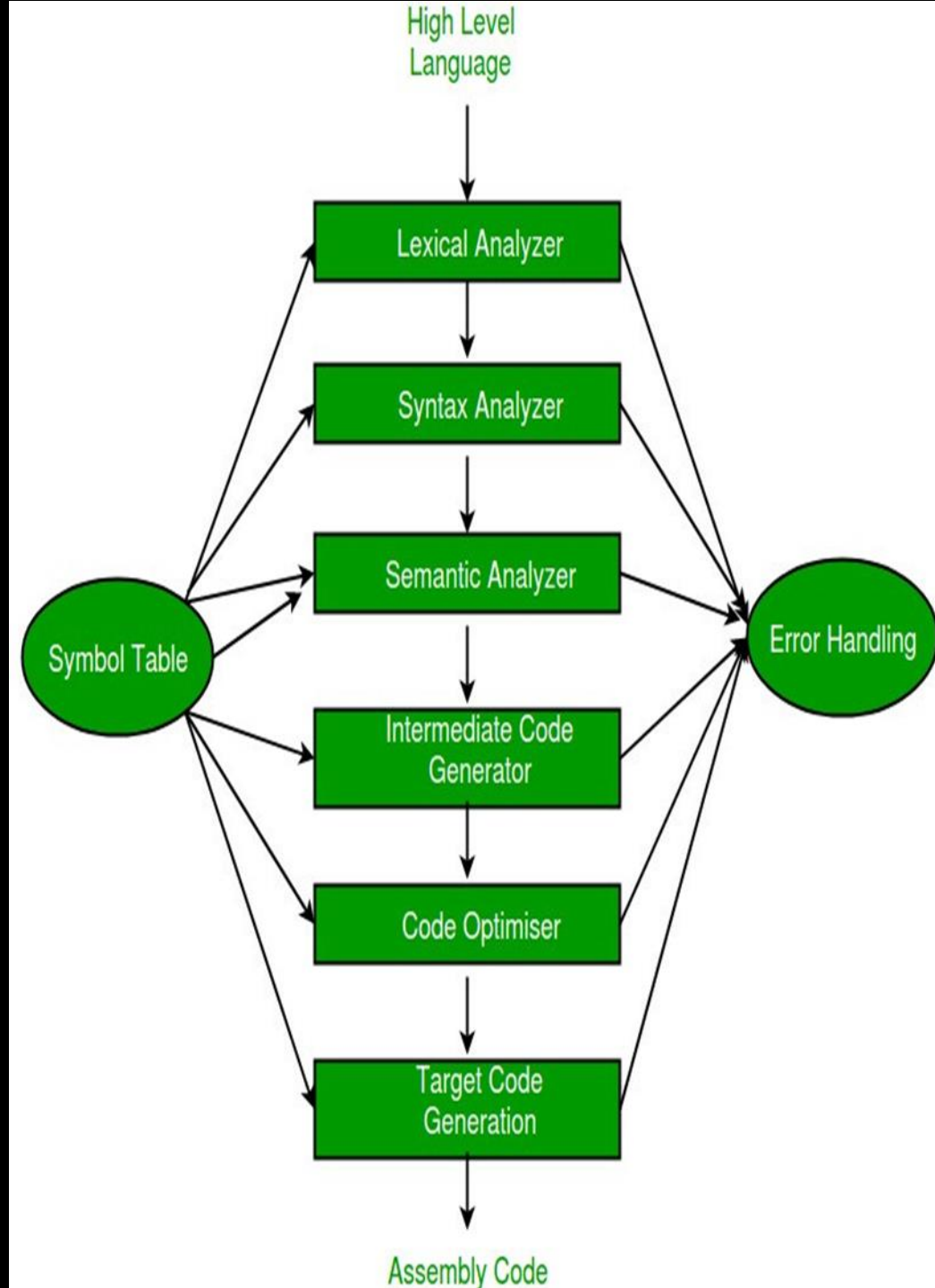


Symbol Table

- It is a data structure being used and maintained by the compiler, consists all the identifier's name along with their types.
- It helps the compiler to function smoothly by finding the identifiers quickly.



- The compiler has two modules namely front end and back end. Front-end constitutes of the Lexical analyser, semantic analyser, syntax analyser and intermediate code generator. And the rest are assembled to form the back end.
- In general Front end fill symbol table and back end uses it.

(1) int x = 10;

Line No	Keyword	identifier	Constant	Operator
1	int	x	10	;

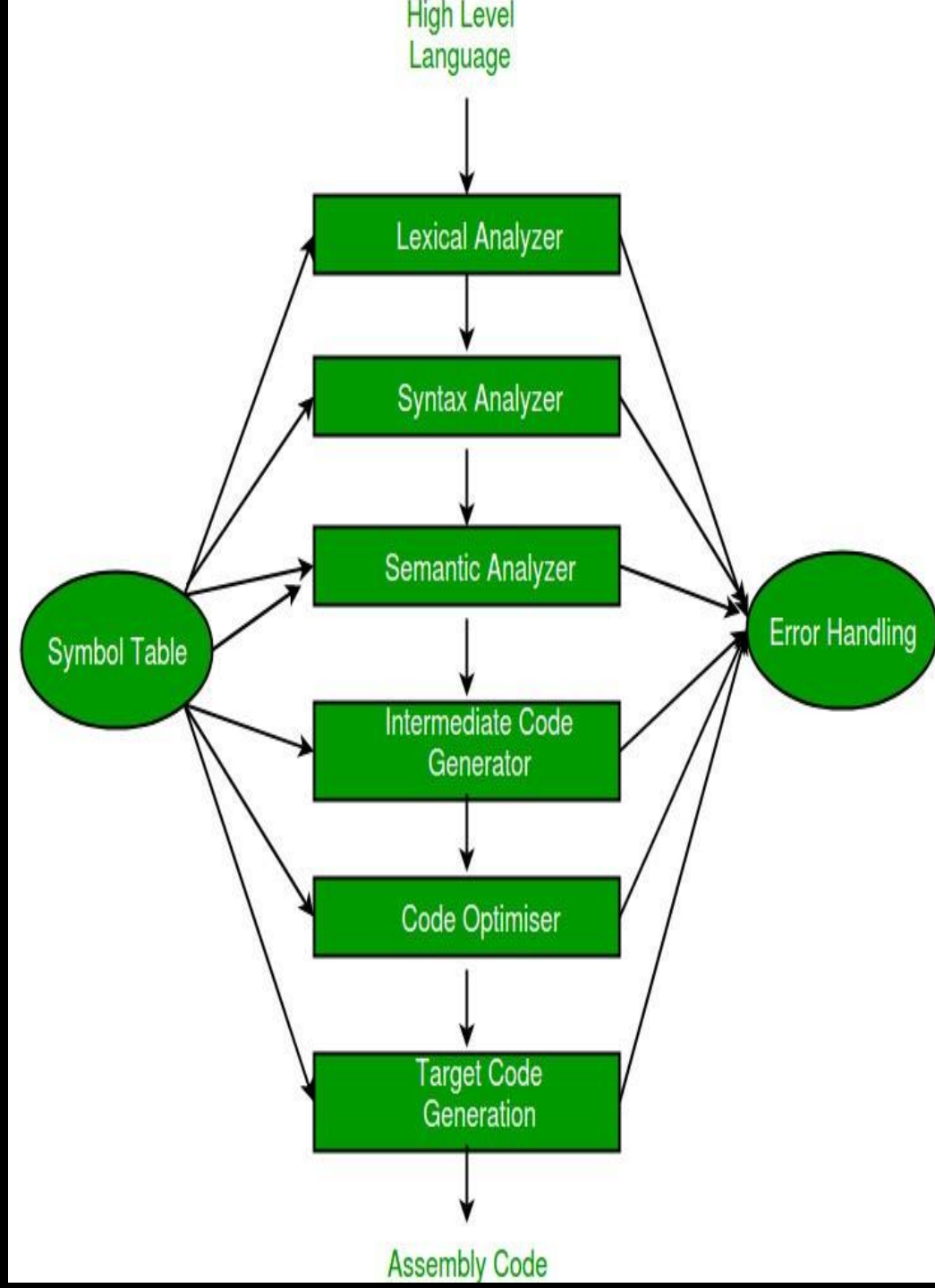
- Lexical analysis in the first phase to communicate with the symbol and the compiler generate the symbol table during the lexical analysis phase.
- Compiler is responsible to provide the memory for symbol table. at every phase if any new variable occurs, then they will be stored in the symbol table.
- Every phase of the compiler will be interacting with the symbol table.
- In general, during the first two phases, we store the information in the symbol table and in the memory and in the later phases, we make use of the information available in symbol table.

- Information stored in the symbol table about identifier
 - name
 - type
 - scope
 - size
 - offset
- Other information in case of array, records and procedures etc.
 - array → size
 - records → column names
 - procedure → i/p parameter
 - functions → i/p, o/p parameter, actual, formal parameter

- Operation on the symbol table: - there are 4 operation function that can be performed on symbol table
 - insert
 - lookup/search
 - Modify
 - delete
 - Implementation of symbol tables can be done using anyone of the data structure
 - liner table
 - Binary Search Tree
 - Linked List
 - Hash Table (
-

Phases and Passes

- A compiler can have many phases and passes.
- **Pass:** A pass refers to the traversal of a compiler through the entire program.
- **Phase:** A phase of a compiler is a distinguishable stage, which takes input from the previous stage, processes and yields output that can be used as input for the next stage. A pass can have more than one phase.



Single Pass Compiler

- **One-Time Processing**: Reads and processes the source code in a single pass, translating it directly into machine code.
- **Speed Advantage**: Faster than multi-pass compilers due to the single traversal of the source code.
- **Memory Efficient**: Uses less memory as it doesn't store extensive intermediate state information.
- **Optimization Limitations**: Offers limited optimization capabilities due to less context understanding of the entire program.
- **Best for Simple Languages**: Ideal for simpler programming languages where complex analysis and deep optimization are not required.
- **Early FORTRAN Compilers, Pascal (Original Version), Tiny C Compiler (TCC)**

Multi-Pass Compiler

- A multi-pass compiler is a type of compiler that processes the source code multiple times before producing the final output. Here's a quick rundown in bullet points:
 - **Multiple Scans of Source Code**: It processes the source code in several passes, each for a specific analysis or transformation task.
 - **Advanced Optimization**: Multi-pass compilers perform complex optimizations by analyzing and re-analyzing the code, improving performance or minimizing resource usage.
 - **Context Awareness**: They have a deeper understanding of the program's context, which allows for more sophisticated error checking and optimization.
 - **Memory Usage**: They typically require more memory than single-pass compilers, as they need to maintain intermediate representations of the source code between passes.
 - **Examples**: Modern compilers like GCC (GNU Compiler Collection) and LLVM are multi-pass compilers that provide extensive optimization and support for complex language features.

Aspect	Single-Pass Compiler	Multi-Pass Compiler
Compilation Speed	Generally faster as it compiles the source code in one pass.	Slower, as it goes through the source code multiple times.
Memory Usage	Less memory-intensive because it doesn't store much intermediate data.	More memory-intensive due to the need to store intermediate representations.
Error Detection	May not detect all errors in the first pass.	Better at error detection, as it analyzes the code in more depth over multiple passes.
Optimization	Limited optimization opportunities due to the single pass nature.	Better optimization, as multiple passes allow for more analysis and refinement.
Language Complexity	Suited for simpler languages with less complex syntax and semantics	Better for complex languages, as it can handle intricate syntax and semantics effectively

DETERMINISTIC FINITE AUTOMATA

A **deterministic finite automaton (DFA)** is defined by 5-tuple $(Q, \Sigma, \delta, S, F)$ where:

- Q is a finite and non-empty set of states
- Σ is a finite non-empty set of finite input alphabet
- δ is a transition function, ($\delta: Q \times \Sigma \rightarrow Q$)
- S is **initial state** (always one) ($S \in Q$)
- F is a set of final states ($F \subseteq Q$) ($0 \leq |F| \leq N$, where n is the number of states)

NFA and DFA Equivalence

- In this topic we will be learning about the equivalence of NFA and DFA and how an NFA can be converted to equivalent DFA. Let us take an example and understand the conversion.
- Since every NFA and DFA has equal power that means, for every language if a NFA is possible, then DFA is also possible.
- So, every NFA can be converted to DFA.
- The process of conversion of an NFA into a DFA is called Subset Construction.
- If NFA have 'n' states which is converted into DFA which 'm' states then the relationship between n and m will be
- $1 \leq m \leq 2^n$

Procedure for Conversion

- There lies a fixed algorithm for the NFA and DFA conversion. Following things must be considered
 - Initial state will always remain same.
 - Start the construction of δ' with the initial state & continue for every new state that comes under the input column and terminate the process whenever no new state appears under the input column.
 - Every subset of states that contain the final state of the NFA is a final state in the resulting DFA.
 - $\delta'(q_0, q_1, q_2, q_3, \dots, q_{n-1}, a) = \bigcup_{i=0}^{n-1} \delta(q_i, a)$

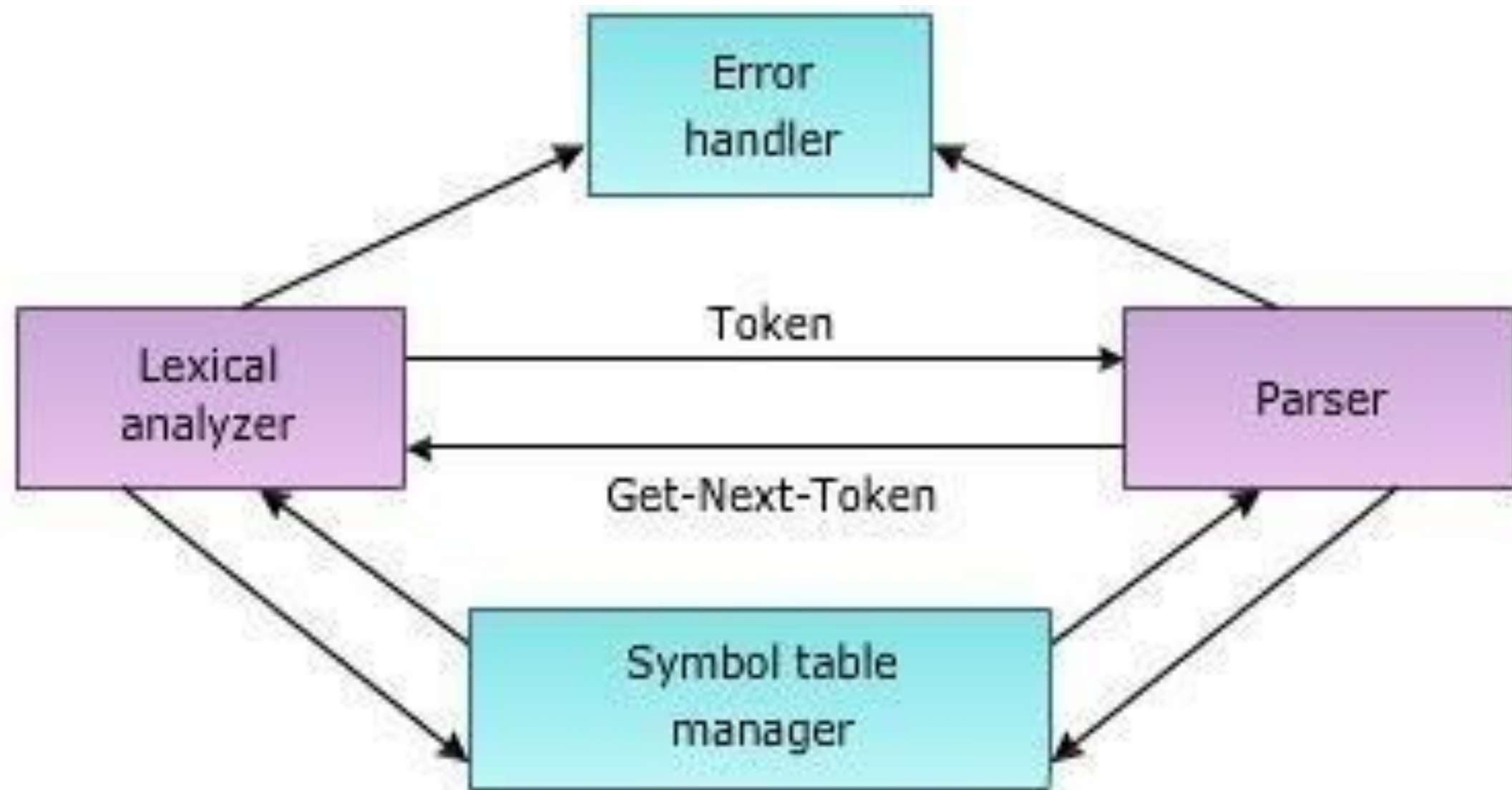
Aspect	DFA (Deterministic Finite Automata)	NDFA (Nondeterministic Finite Automata)
State Transition	On each input symbol, transitions to exactly one state.	Can transition to multiple states or none on the same input symbol.
Determinism and Uniqueness	Each state has a unique transition for each input symbol.	A state can have multiple transitions for the same input symbol.
Computation Path	Always has a single, unique computation path for any input string.	May have multiple computation paths for the same input string.
Ease of Construction	Generally simpler and more straightforward to construct.	Can be more complex to construct due to non-determinism.
Acceptance of Input	Accepts an input if it reaches a final state after processing all input	Accepts an input if at least one computation path reaches a final state.

LEXICAL ANALYZER

- **LEXICAL ANALYSIS** is the very first phase in the compiler designing.
- takes the modified source code which is written in the form of sentences . In other words, it helps you to convert a sequence of characters into a sequence of tokens.
- The lexical analyzer breaks this syntax into a series of tokens. It removes any extra space or comment written in the source code.
- Programs that perform lexical analysis are called lexical analyzers or lexers.
- A lexer contains tokenizer or scanner. If the lexical analyzer detects that the token is invalid, it generates an error.
- It reads character streams from the source code, checks for legal tokens, and pass the data to the syntax analyzer when it demands.

- Lexical Analysis is the first phase of the compiler also known as a scanner. It converts the High level input program into a sequence of **Tokens**.
- Lexical Analysis can be implemented with the Deterministic finite Automata.
- The output is a sequence of tokens that is sent to the parser for syntax analysis.
- It is the only phase in the compiler which reads code character by character and generate tokens.

- Lexical Analyzer is the first phase of compiler
- Concept/Process is Tokenization.
- Source prog is INPUT.
- Parser request for next Token
- LA provide token to parser.
- The main task of lexical analyzer (Scanner) is to read a stream of characters as an input and produce a sequence of tokens that the syntax analyzer (Parser) uses for syntax analysis.
- It is the only phase in the compiler which reads code character by character and generate tokens.



- Tokenization i.e. Dividing the program into valid tokens.
- Remove white space characters.
- Remove comments.
- It also provides help in generating error messages by providing row numbers. (syntax logical error).
- A). Exceeding length: exceeding length in variable/function assignment.
- B). Unmatched String: `return b; #` (illegal character use), comment start but not end.
- Identify tokens and place them in symbol table.

Lexical Analyzer

- A *lexeme* is a sequence of characters in the source program that matches the pattern for a token and is identified by the lexical analyzer as an instance of that token. Where a token is a strings with an assigned and thus identified meaning. Actual representation or stream of characters is called as Lexemes; Logical meaning of Lexemes is known as Tokens.
- Lexing can be divided into two stages: the *scanning*, which segments the input string into syntactic units called *lexemes* and categorizes these into token classes; and the *evaluating*, which converts lexemes into processed values.

Lexeme	token
while	while
(	lparen
y	identifier
>=	Comparison
t	identifier
)	Rparen
y	identifier
=	Assignment
y	identifier
-	Arithmetic
3	integer
;	Finish of a statement

Tokens

Lexemes are said to be a sequence of characters (alphanumeric) in a token.

Tokens in compiler design are the sequence of characters which represents a unit of information in the source program.

There are some predefined rules for every lexeme to be identified as a valid token. These rules are defined by grammar rules, by means of a pattern.

A pattern is a description which is used by the token. In the case of a keyword which uses as a token, the pattern is a sequence of characters. A pattern explains what can be a token, and these patterns are defined by means of regular expressions.

In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.

For example, in C language, the variable declaration line

int value = 100;

contains the tokens:

int (keyword), value (identifier), = (operator), 100 (constant) and ; (symbol).

Specifications of Tokens

Let us understand how the language theory undertakes the following terms:

Alphabets

Any finite set of symbols $\{0,1\}$ is a set of binary alphabets, $\{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$ is a set of Hexadecimal alphabets, $\{a-z, A-Z\}$ is a set of English language alphabets.

Strings

Any finite sequence of alphabets is called a string. Length of the string is the total number of occurrence of alphabets, e.g., the length of the string Pakistan is 8 and is denoted by $| \text{Pakistan} | = 8$. A string having no alphabets, i.e. a string of zero length is known as an empty string and is denoted by ϵ (epsilon).

Special Symbols

A typical high-level language contains the following symbols:-

Arithmetic Symbols	Addition(+), Subtraction(-), Modulo(%), Multiplication(*), Division(/)
Punctuation	Comma(,), Semicolon(;), Dot(.), Arrow(->)
Assignment	=
Special Assignment	+=, /=, *=, -=
Comparison	==, !=, <, <=, >, >=
Preprocessor	#
Location Specifier	&
Logical	&, &&, , , !
Shift Operator	>>, >>>, <<, <<<

- **Lexeme**: A lexeme is a sequence of characters that are included in the source program according to the matching pattern of a token. It is nothing but an instance of a token.

For example **(count=count+temp;)**

2 variables count and temp

operators + = (;) punctuation

Token types identifier, operator and punctuation(;;)

Here count and temp are lexemes of Id.

- **Pattern**: A pattern is a description which is used by the token. In the case of a keyword which uses as a token, the pattern is a sequence of characters.
- A rule that describes the set of strings associated to a token. Expressed as a regular expression and describing how a particular token can be formed. (like identifier start with alphabet, followed by alphanumeric character but not keyword/Digit).
- **Token**: The token is a sequence of characters which represents a unit of information in the source program.

Lexeme	Pattern	Token
a, sum, Total	Starting with letter and followed by letter, digit but not keyword	ID
If Then Else, for, while etc..	If Then Else, for, while	Keywords /Reserved words
123, 123.4, 12e+07	Starting with digit followed by digit or optional fraction or exponent	Num
< , > , <> , <= , >= , =	< or > or <> or <= or >= or =	Relop
“Lateral String”	Any sequence of character within quotations	String
, ; () { }	, or ; or (or) or { or }	Comma, semicolon, open bracket, close bracket or punctuation

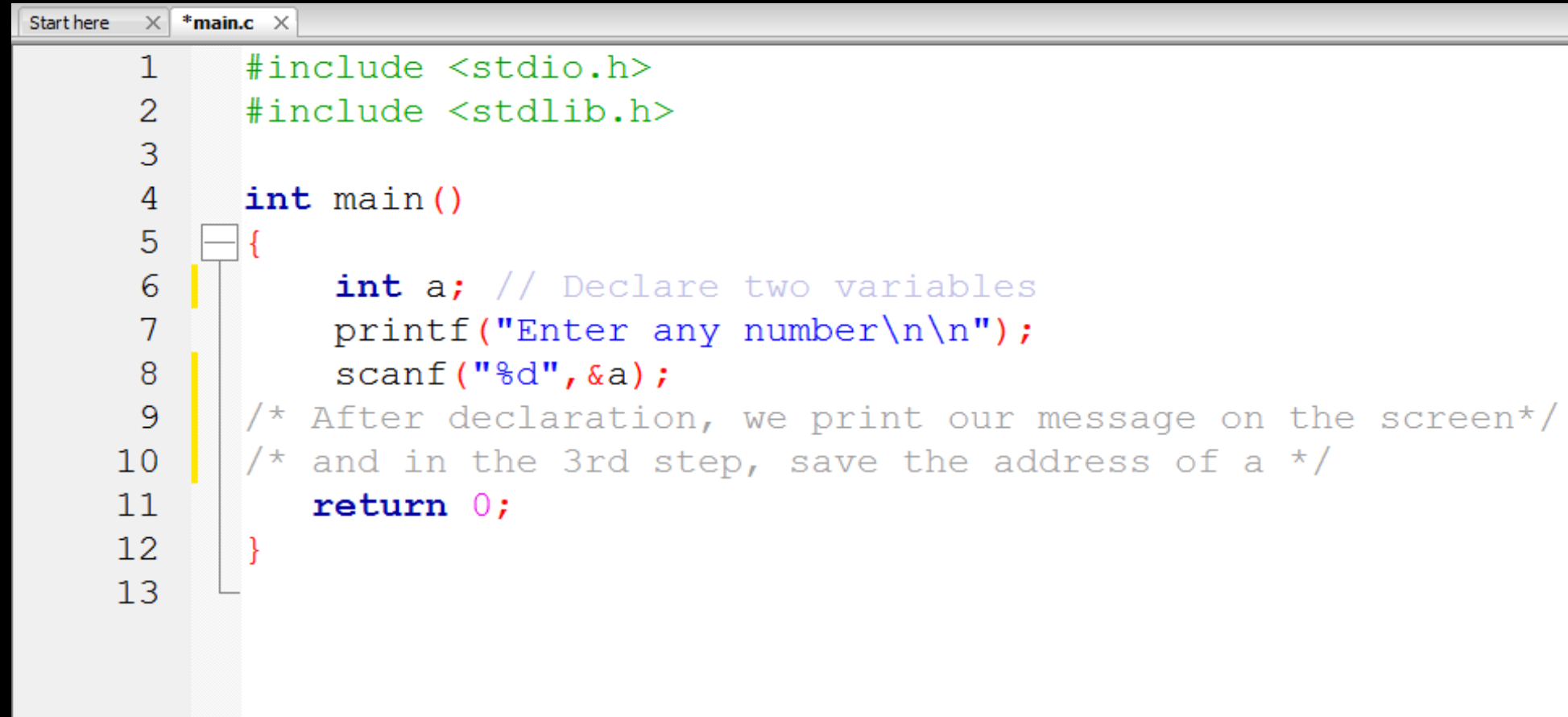
Lexical Analyzer Architecture: How tokens are recognized

- The main task of lexical analysis is to read input characters in the code and produce tokens.
- Lexical analyzer scans the entire source code of the program. It identifies each token one by one. Scanners are usually implemented to produce tokens only when requested by a parser.
 1. "Get next token" is a command which is sent from the parser to the lexical analyzer.
 2. On receiving this command, the lexical analyzer scans the input until it finds the next token.
 3. It returns the token to Parser.
- Lexical Analyzer skips whitespaces and comments while creating these tokens. If any error is present, then Lexical analyzer will correlate that error with the source file and line number.

Roles of the Lexical analyzer

- Lexical analyzer performs below given tasks:
 1. Helps to identify token into the symbol table
 2. Removes white spaces and comments from the source program
 3. Correlates error messages with the source program
 4. Helps you to expands the macros if it is found in the source program
 5. Read input characters from the source program
 6. It also provides help in generating error messages by providing row numbers and column numbers.

- Secondary Function of Lexical Analyser are: -
 - Removal of Comments lines



```
Start here x *main.c x
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main()
5  {
6      int a; // Declare two variables
7      printf("Enter any number\n\n");
8      scanf("%d",&a);
9      /* After declaration, we print our message on the screen*/
10     /* and in the 3rd step, save the address of a */
11     return 0;
12 }
13
```

- Removal of White space characters

WHITE SPACE CHARACTERS

\b	blank space	\t	horizontal tab	\v	vertical tab
\\	Back slash	\'	Single quote	\"	Double quote
\r	carriage return	\f	form feed	\n	new line
\?	Question mark	\0	Null	\a	Alarm (bell)

- Co-relating with Error Messages along with line number.

```
File Edit Search Run Compile Debug Project Options Window Help
\TC\SS.C
#include<stdio.h>
#include<conio.h>
void main()
{
int sum=0,n1,n2;
a=0;
clrscr();
printf("enter two number= ");
scanf("%d%d",&n1,&n2);
sum=n1+n2;
printf("Sum =: %d",sum);
getch();
}
```

Compile time error here
a is not defined but it is
used in program

Error ..\SS.C 6: Undefined symbol 'a'

6:3

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

Lexical Analyzer vs. Parser

Lexical Analyser	Parser
Scan Input program	Perform syntax analysis
Identify Tokens	Create an abstract representation of the code
Insert tokens into Symbol Table	Update symbol table entries
It generates lexical errors	It generates a parse tree of the source code

Why separate Lexical and Parser?

- The simplicity of design: It eases the process of lexical analysis and the syntax analysis by eliminating unwanted tokens
- To improve compiler efficiency: Helps you to improve compiler efficiency
- Specialization: specialized techniques can be applied to improves the lexical analysis process
- Portability: only the scanner requires to communicate with the outside world

- **Advantages of Lexical analysis**

- Lexical analyzer method is used by programs like compilers which can use the parsed data from a programmer's code to create a compiled binary executable code
- It is used by web browsers to format and display a web page with the help of parsed data from JavaScript, HTML, CSS
- A separate lexical analyzer helps you to construct a specialized and potentially more efficient processor for the task

Match Rule in LA

- Lexical analyzer read character by character and when the white space or a operator or special symbol occur it stops and consider these characters as a word.

□ For Example:

- `int a ;` (`int` keyword , `a` id1 and `;`semicolon)
- `inta ;` (`inta` id1 and `;`semicolon)
- `a++ ;` (`a` id1 with increment operator `++` and `;`semicolon)

SYMBOL TABLE

- Symbol table stores tokens.
- Token <Class part ,Value part>
- For Example
 - `int c ;`

SYMBOL TABLE	
Class Part	Value Part
keyword	int
id1	c
semicolon	;

TOKEN SET

- Token set contains line number ,class part and value part.

- For Example

 - **int value = 100;**

- *Token Set:*

- (keyword, int) (id1, value) (eq-op, =) (num1, 100) (semicolon, ;) **Line**

1

Example

•Create symbol table and token set for the given lines of code:

- `int c, a, b ;`
- `c = a + b * 5 ;`

TOKEN SET

(keyword, int) (id1, c) (id2, a) (id3, b) (semicolon, ;) **Line 1**

(id1, c) (eq-op, =) (id2, a) (add-op, +) (id3, b) (mul-op, *) (num1, 5) (semicolon, ;) **Line 2**

SYMBOL TABLE

Class Part	Value Part
keyword	int
id1	c
id2	a
id3	b
num1	5
semicolon	;
eq-op	=
add-op	+
mul-op	*

Examples:

1) Comments are neglected (not count).

2). int a = 20, b = 30; No. of tokens = 9.

3). printf ("i = %d, &i = %x", i, &i); No. of tokens = 10

4). int main)(No. of tokens = 4.
(LA, wek is only to convert into tokens
making sense is not (is wek).

5) printf ("%d %d, a, b); no closing " we stop here
(Double quote) & stop the counting in this
step.

6) For some symbols you have to ~~check~~
check next symbol as symbol (+ is unary as
well as binary operator.
++ , count as single. (encounter + check its next
symbol as well).
+= , --

7). in / * comment * / + m = 40.5 ; No. of tokens = 6.
in and + space is here, count as
different token.
in / * comment * / + → gap.

- Suppose we pass a statement through lexical analyzer –

`a = b + c ;` It will generate token sequence like this:

`id=id+id;` Where each id refers to it's variable in the symbol table
referencing all details

- For example, consider the program

```
int main()  
{  
    // 2 variables  
    int a, b;  
    a = 10;  
    return 0;  
}
```

All the valid tokens are:

```
'int' 'main' '(' ')' '{' 'int' 'a' ',' 'b' ';' 'a' '=' '10' ';' 'return' '0' ';' '}'
```


Exercise 1: (Task)

Count number of tokens :

```
int main()
{
    int a = 10, b = 20;
    printf("sum is :%d",a+b);
    return 0;
}
```

Exercise 2:

Count number of tokens :

- `int max(int i);`
- `int, max, ( ,int, i, ), , ;`

TASK: Identify number of tokens in the following code

□ Example#1

➤ `int i = 2000 ;`

➤ ?? tokens

▪ Example#2

➤ `a++ + b ;`

➤ ?? tokens

□ Example#3

➤ `int max ( i ,j){ int i ,j;`

• `/* return max of i and j*/`

• `return ( i > j? i !j );`

• `}`

➤ ?? tokens

- Let's consider the following C program given below to understand the working.

```
int main()
{
    x = y+z;
    int x, y, z;
    print("Goto GFG %d%d", a);
    {
```

```
|int | main |) | (
| } |
|x| = |y| + |z| ;| |
|int| x| ,| y| ,| z| ;|
|print| ( | "Goto GFG %d%d" | ,| a| ) | ;|
|{|
```

Will count as 1

Total Token = 26

- In the first phase, the compiler doesn't check the syntax. So, here this program is input to the lexical analyzer and converted into tokens. So, tokenization is one of the important functions of the lexical analyzer.
- The total number of tokens for this program is 26. Below is the diagram of how it will count the tokens.

Symbols and Operators

- Arithmetic Operators (+ - % * /)
- Punctuation Symbols (, ; . () { } [])
- Logical Operators (& ! && ||)
- Assignment Operators (= += /= -= *=)
- Pre-processor (#)
- Shift Operators (>> >>> <<)
- Exponent (^)
- Identifier (**Starting with letters followed by some letter or number but not keyword**).
- Keywords (**if then for while else etc**)

DESIGN CONSIDERATION FOR OF LEXICAL ANALYZER

- **Free Format Input**

- The lexical analyzer only checks the lexeme and generate tokens.

- **Blanks Significance**

- Blanks are the most significance in designing a lexical analyzer White spaces are the identification of lexeme ends.

- **int a**

- **inta**

- **Keywords must be reserved**

- **Longest match approach**

- Longest Match Rule

- When the lexical analyzer reads the source-code, it scans the code letter by letter; and when it encounters a whitespace, operator symbol, or special symbols, it decides that a word is completed.

For example:

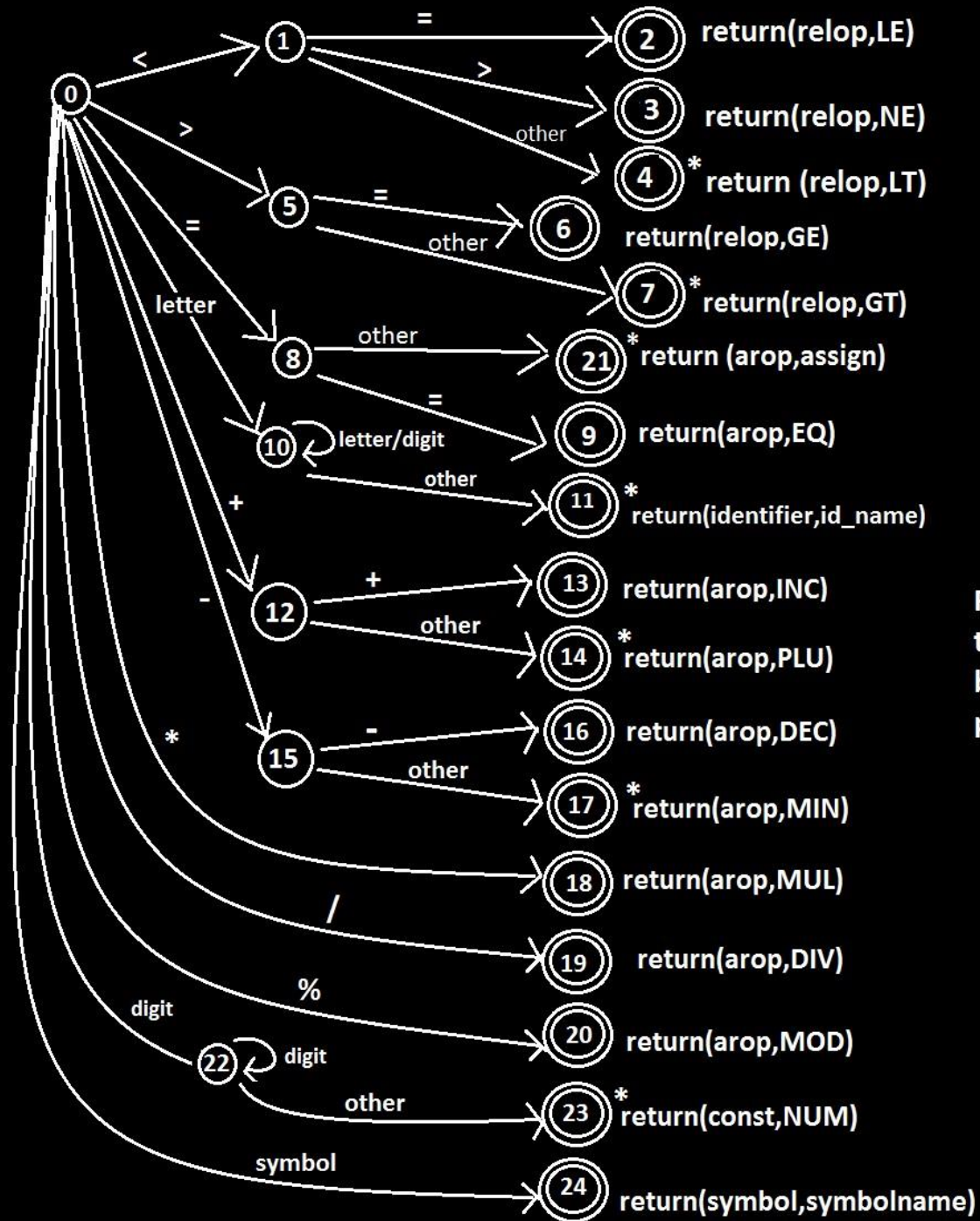
int intvalue;

- While scanning both lexemes till 'int', the lexical analyzer cannot determine whether it is a keyword int or the initials of identifier int value.
- The Longest Match Rule states that the lexeme scanned should be determined based on the longest match among all the tokens available.
- The lexical analyzer also follows rule priority where a reserved word, e.g., a keyword, of a language is given priority over user input. That is, if the lexical analyzer finds a lexeme that matches with any existing reserved word, it should generate an error.

Implementation of Lexical analyzer

Lexical analyzer can be implemented in following step :

- Input to the lexical analyzer is a source program.
- By using input buffering scheme, it scans the source program.
- Regular expressions are used to represent the input patterns.
- Now this input pattern is converted into NFA by using finite automation machine.
- This NFA are then converted into DFA and DFA are minimized by using different method of minimization.
- The minimized DFA are used to recognize the pattern and broken into lexemes.
- Each minimized DFA is associated with a phase in a programming language which will evaluate the lexemes that match the regular expression.
- The tool then constructs a state table for the appropriate finite state machine and creates program code which contains the table, the evaluation phases, and a routine which uses them appropriately.



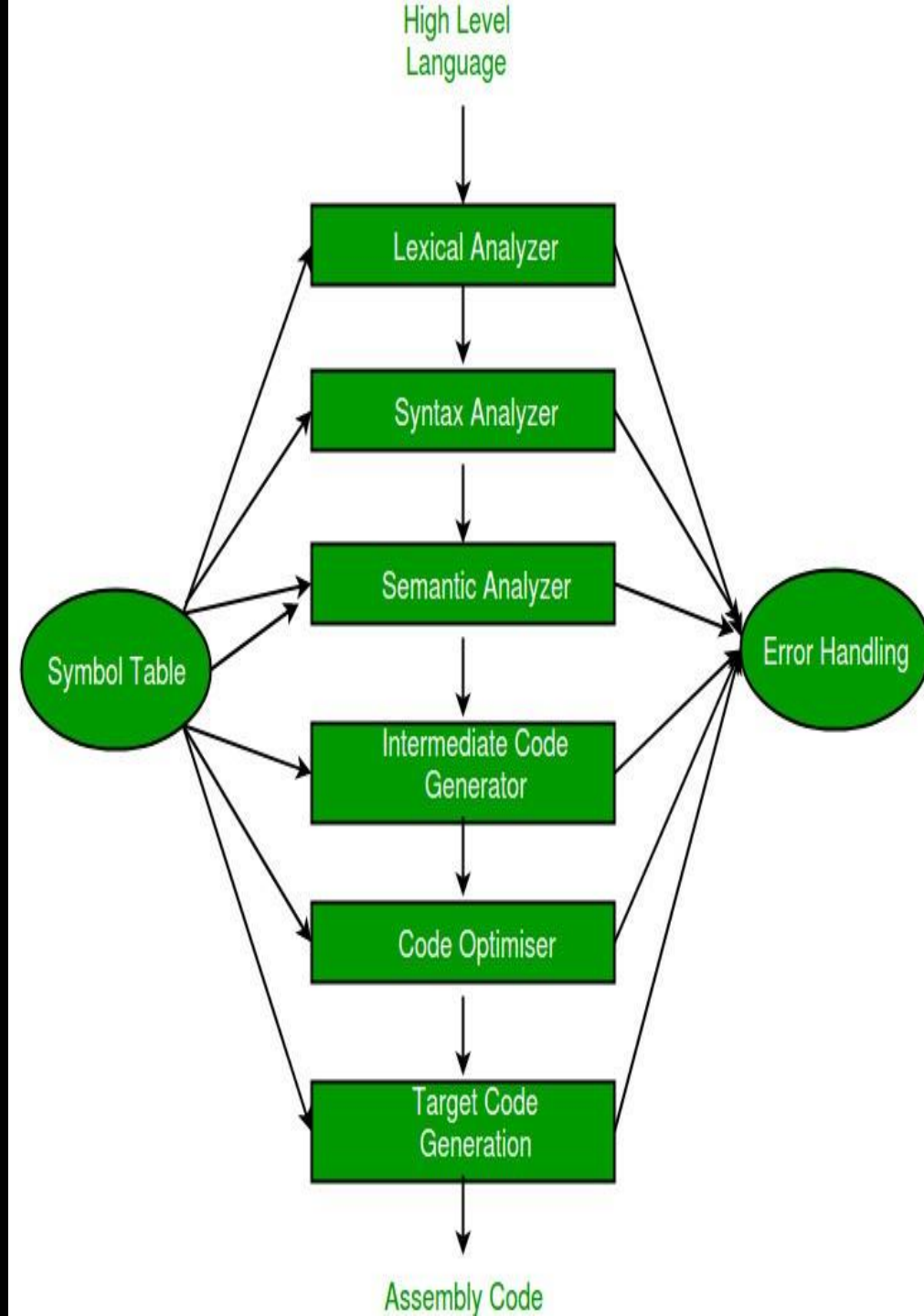
Final states marks with * requires to the forward pointer (fwdptr) to be retracted by one character position

Lexical Errors

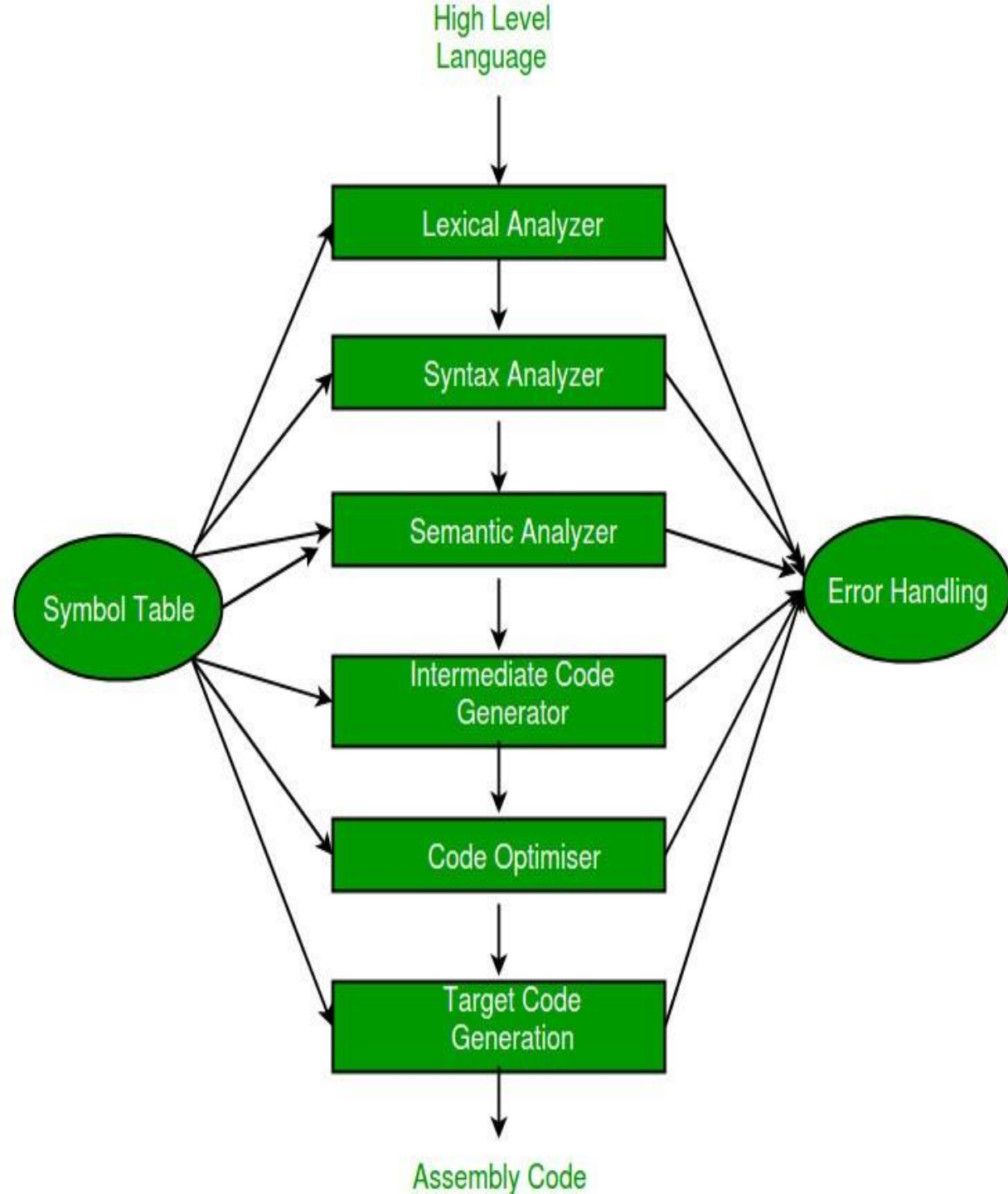
- A character sequence which is not possible to scan into any valid token is a lexical error. Important facts about the lexical error:
 1. Lexical errors are not very common, but it should be managed by a scanner
 2. Misspelling of identifiers, operators, keyword are considered as lexical errors
 3. Generally, a lexical error is caused by the appearance of some illegal character, mostly at the beginning of a token.

Error handler

- It is a sub routine to care take care of the continuation of compilation, even any error at any phase, i.e. error handler is responsible to continue the compilation process even any error occurs at phase 1 or phase 2 or phase 3.
- After phase 3, if the error handler object is empty, then the source code is free error and it can be converted into target code. if the error handler object is not empty after phase 3 then there will be some error at phase 1,2 or 3 then the error will be displayed.



- Compiler can handle, there types of errors: -
 - lexical error
 - syntax error
 - semantic error
- The error handles by compiler are known as exception and programmer is responsible to handle the exception.
- At the time of execution also, we can get some error they are called fatal error and system admin is responsible to handle fatal error.



Error Handling in Compiler Design

- The tasks of the **Error Handling** process are to detect each error, report it to the user, and then make some recover strategy and implement them to handle error. During this whole process processing time of program should not be slow. An **Error** is the blank entries in the symbol table.
- **Types or Sources of Error**
- A **run-time error** is an error which takes place during the execution of a program, and usually happens because of adverse system parameters or invalid input data. The lack of sufficient memory to run an application or a memory conflict with another program and logical error are example of this. Logic errors, occur when executed code does not produce the expected result. Logic errors are best handled by meticulous program debugging.
- **Compile-time errors** rises at compile time, before execution of the program. Syntax error or missing file reference that prevents the program from successfully compiling is the example of this.

Classification of Compile-time error –

- **Lexical** : This includes misspellings of identifiers, keywords or operators
- **Syntactical** : missing semicolon or unbalanced parenthesis
- **Semantical** : incompatible value assignment or type mismatches between operator and operand
- **Logical** : code not reachable, infinite loop.

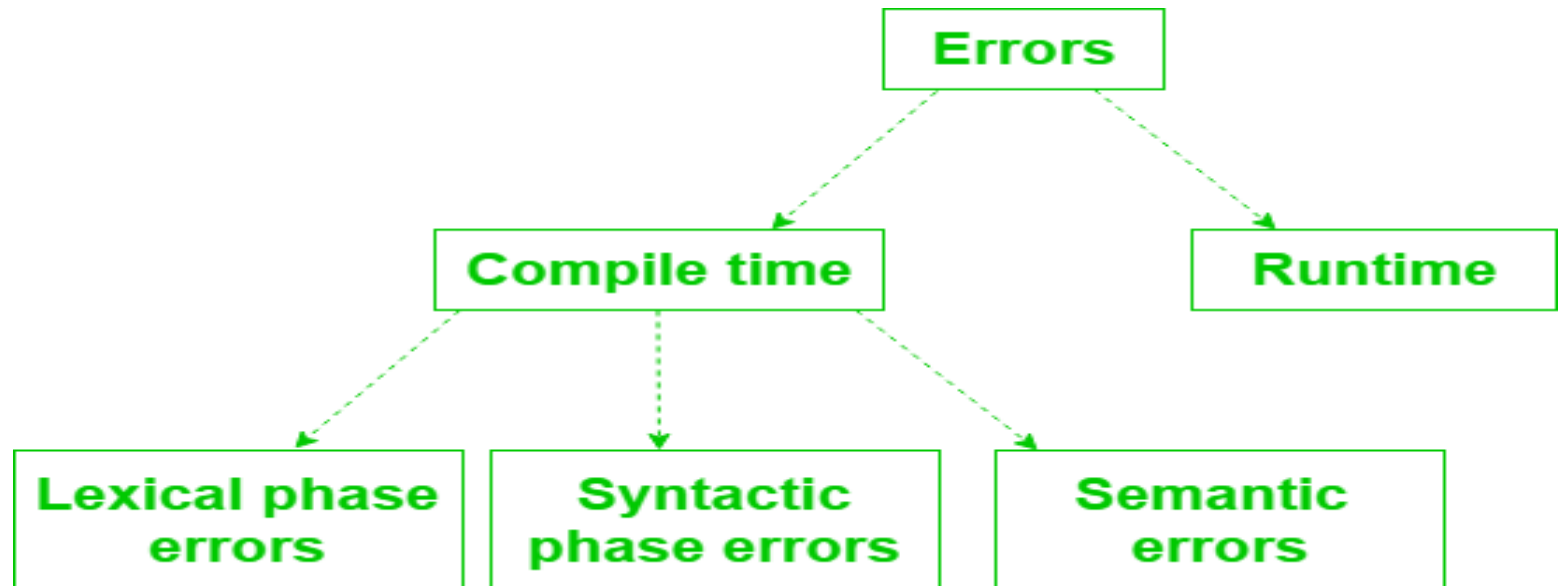
- Error detection and Recovery in Compiler

In this phase of compilation, all possible errors made by the user are detected and reported to the user in form of error messages. This process of locating errors and reporting it to user is called **Error Handling process**.

Functions of Error handler

- Detection
- Reporting
- Recovery

- Classification of Errors



Lexical phase errors

These errors are detected during the lexical analysis phase. Typical lexical errors are

- **Exceeding length of identifier or numeric constants.**
- **Appearance of illegal characters**
- **Unmatched string**
- **Example 1 : `printf("COMPILER");$`**

This is a lexical error since an illegal character \$ appears at the end of statement.

- **Example 2 : This is a comment */**
- This is an lexical error since end of comment is present but beginning is not present.

Error recovery:

Panic Mode Recovery

- In this method, successive characters from the input are removed one at a time until a designated set of synchronizing tokens is found. Synchronizing tokens are delimiters such as ; or }
- Advantage is that it is easy to implement and guarantees not to go to infinite loop
- Disadvantage is that a considerable amount of input is skipped without checking it for additional errors

Errors in Lexical Analysis & their Recovery

- Types of Errors detected by Lexical Analyzer

1. Long Identifiers
2. Too long Numerical literals
3. Badly formed numerical literals
4. Input characters that are not present in source language
5. Spelling Mistakes

Error Recovery

- Delete (Panic-mode recovery)->> Unknown characters are deleted.
- Insert-> Extra or missing character is inserted.
- Transpose
- Replace

Error Recovery in Lexical Analyzer

- 1) Removes one character from the remaining input
- 2) In the panic mode, the successive characters are always ignored until we reach a well-formed token
- 3) By inserting the missing character into the remaining input
- 4) Replace a character with another character
- 5) Transpose two serial characters